

订单延时关闭功能技术选型

定时任务

通过定时任务是一种常见的订单延迟关闭解决方案。

可以通过调度平台来实现定时任务的执行，具体任务是根据订单创建时间扫描所有到期的订单，并执行关闭订单的操作。

大家常用的定时任务调度平台有以下这些：

- <<https://github.com/xuxueli/xxl-job>>
- <<https://github.com/PowerJob/PowerJob>>
- <<https://github.com/apache/shardingsphere-elasticjob>>

这种方案的优点在于简单易实现，但是，该方案也存在一些问题：

1. 延迟时间不精确：使用定时任务执行订单关闭逻辑，无法保证订单在十分钟后准确地关闭。如果任务执行器在关闭订单的具体时间点出现问题，可能导致订单关闭的时间延后。
2. 不适合高并发场景：定时任务执行的频率通常是固定的，无法根据实际订单的情况来灵活调整。在高并发场景下，可能导致大量的定时任务同时执行，造成系统负载过大。
3. 分库分表问题：拿 12306 来说，订单表按照用户标识和订单号进行了分库分表，那这样的话，和上面说的根据订单创建时间去扫描一批订单进行关闭，自然就行不通。因为根据创建时间查询没有携带分片键，存在读扩散问题。

通常最不推荐的方式是使用定时任务来实现订单关闭。

RabbitMQ



项目地址：<<https://github.com/rabbitmq/rabbitmq-server>>

RabbitMQ 是一个功能强大的消息中间件，通过使用 RabbitMQ 的延时消息特性，我们可以轻松实现订单十分钟延时关闭功能。首先，我们需要在 RabbitMQ 服务器上启用延时特性，通常

通过安装 `rabbitmq_delayed_message_exchange` 插件来支持延时消息功能。

接下来，我们创建两个队列：订单队列和死信队列。订单队列用于存储需要延时关闭的订单消息，而死信队列则用于存储延时时间到达后的订单消息。在创建订单队列时，我们要为队列配置延时特性，指定订单消息的延时时间，比如十分钟。这样，当有新的订单需要延时关闭时，我们只需要将订单消息发送到订单队列，并设置消息的延时时间。

在订单队列中设置死信交换机和死信队列，当订单消息的延时时间到达后，消息会自动转发到死信队列，从而触发关闭订单的操作。在死信队列中，我们可以监听消息，并执行关闭订单的逻辑。为了确保消息的可靠性，可以在关闭订单操作前添加适当的幂等性措施，这样即使消息重复处理，也不会对系统产生影响。

通过以上步骤，我们就成功实现了订单的十分钟延时关闭功能。当有新的订单需要延时关闭时，将订单消息发送到订单队列，并设置延时时间。在延时时间到达后，订单消息会自动进入死信队列，从而触发关闭订单的操作。这种方式既简单又可靠，保证了系统的稳定性和可用性。

从整体来说 RabbitMQ 实现延时关闭订单功能是比较合适的，但也存在几个问题：

1. 延时精度：RabbitMQ 的延时消息特性是基于消息的 TTL (Time-To-Live) 来实现的，因此消息的延时时间并不是完全准确的，可能会有一定的误差。在处理订单十分钟延时关闭时，可能会有一些订单的关闭时间略晚于预期时间。
2. 高并发问题：如果系统中有大量的订单需要延时关闭，而订单关闭操作非常复杂耗时，可能会导致消息队列中的消息堆积。这样就可能导致延时关闭操作无法及时处理，影响订单的实际关闭时间。
3. 重复消息问题：由于网络原因或其他不可预知的因素，可能会导致消息重复发送到订单队列。如果没有处理好消息的幂等性，可能会导致订单重复关闭的问题，从而造成数据不一致或其他异常情况。
4. 可靠性问题：RabbitMQ 是一个消息中间件，它是一个独立的系统。如果 RabbitMQ 本身出现故障或宕机，可能会导致订单延时关闭功能失效。因此，在使用 RabbitMQ 实现延时关闭功能时，需要考虑如何保证 RabbitMQ 的高可用性和稳定性。

延时精度和高并发属于一类问题，取决于客户端的消费能力。重复消费问题是所有消息中间件都需要解决，需要通过消息表等幂等解决方案解决。比较难搞定的是可用性问题，RabbitMQ 在可用性方面较弱，部分场景下会存在单点故障问题。

Redis 过期监听



项目地址：<<https://github.com/redis/redis>>

可以借助 Redis 的过期消息监听机制实现延时关闭功能。

首先，在订单创建时，将订单信息存储到 Redis，并设置过期时间为十分钟。同时，在 Redis 中存储一个过期消息监听的键值对，键为订单号，值为待处理订单的标识。

其次，编写一个消息监听器，持续监听 Redis 的过期事件。监听器使用 Redis 的 `PSUBSCRIBE` 命令订阅过期事件，并在监听到过期事件时触发相应的处理逻辑。

当订单过期时间到达时，Redis 会自动触发过期事件，消息监听器捕获到该事件，并获取到过期的订单号。接着，监听器执行订单关闭的逻辑，如更新订单状态为关闭状态，释放相关资源等，实现订单的十分钟延时关闭功能。

需要注意的是，消息监听器应该是一个长期运行的任务，确保持续监听 Redis 的过期事件。为了保证系统的稳定性和可靠性，可以在实现订单关闭逻辑时添加容错机制，以应对 Redis 可能发生故障或重启的情况。

Redis 过期消息也存在几个问题：

1. 不够精确：Redis 的过期时间是通过定时器实现的，可能存在一定的误差，导致订单的关闭时间不是精确的十分钟。这对于某些对时间要求较高的场景可能不适用。
2. Redis 宕机：如果 Redis 宕机或重启，那些已经设置了过期时间但还未过期的订单信息将会丢失，导致这部分订单无法正确关闭。需要考虑如何处理这种异常情况。
3. 可靠性：依赖 Redis 的过期时间来实现订单关闭功能，需要确保 Redis 的高可用性和稳定性。如果 Redis 发生故障或网络问题，可能导致订单关闭功能失效。

4. 版本问题：Redis 5.0 之前是不保证延迟消息持久化的，如果客户端消费过程中宕机或者重启，这个消息不会重复投递。5.0 之后推出了 Stream 功能，有了持久化等比较完善的延迟消息功能。

Redisson



项目地址：<<https://github.com/redisson/redisson>>

通过 Redisson 的 `RDelayedQueue` 功能可以实现订单十分钟延时关闭的功能。

首先，我们需要创建一个 `RDelayedQueue` 对象，用于存放需要延时关闭的订单信息。当用户创建订单时，我们将订单信息添加到 `RDelayedQueue` 中，并设置订单的延时时间为十分钟。

Redisson 提供了监听功能，可以实现对 `RDelayedQueue` 中订单信息的监听。一旦订单到达设定的延时时间，Redisson 会触发监听事件。在监听到订单的延时事件后，我们可以编写相应的处理逻辑，即关闭对应的订单。

在处理订单关闭时，我们可以根据订单号或订单创建时间等信息，来找到对应的订单进行关闭操作。

不过这种方式也不推荐使用，基本上 Redis 过期监听消息存在的问题，`RDelayedQueue` 也都会有，因为 `RDelayedQueue` 本质上也是依赖 Redis 实现。

RocketMQ

**Apache
RocketMQ**



项目地址：<<https://github.com/apache/rocketmq>>

在订单生成时，我们将订单关闭消息发送到 RocketMQ，并设置消息的延迟时间为十分钟。RocketMQ 支持设置消息的延迟时间，可以通过设置消息的 `delayLevel` 来指定延迟级别，每个级别对应一种延迟时间。这样，订单关闭消息将在十分钟后自动被消费者接收到。

需要注意，RocketMQ 5.0 之后已经支持了自定义时间的延迟，而不仅是延迟级别范围内的时间。

为了处理订单关闭消息，我们需要在消费者端创建一个消息监听器。当消息监听器接收到订单关闭消息时，触发订单关闭操作，将订单状态设置为关闭状态。

需要注意的是，RocketMQ 的消息传递机制保证了消息的可靠性传递，因此消息可能会进行多次重试。为了确保订单关闭操作的幂等性，即多次执行不会产生副作用，我们需要在订单关闭逻辑中进行幂等性的处理。

因为 12306 系统中也使用到了 RocketMQ，所以这个解决方案将是咱们的不二选择。

12306 系统中具体代码地址：

```
// 延时关闭订单消息生产者
```

```
org.opengoofy.index12306.biz.ticket.service.mq.produce.AbstractCommonSendProduceTemplate#sendMessage
```

```
// 延时关闭订单消息消费者
```

```
org.opengoofy.index12306.biz.ticket.service.mq.consumer.DelayCloseOrderConsumer#onMessage
```

关于 RocketMQ 4.x 版本和 5.x 版本的延时消息实现原理参考文章：

弥补延时消息的不足，RocketMQ 基于时间轮算法实现了定时消息！

更新: 2023-08-06 10:58:19

原文: <<https://www.yuque.com/magestack/12306/sazb1bn2a1okwtlu>>